

```
# =====
# 1. IMPORT LIBRARIES
# =====
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

import seaborn as sns
import matplotlib.pyplot as plt
```

```
def load_file(file_path):
    rows = []

    with open(file_path, "r") as f:
        for line in f:
            line = line.strip()

            if not line:
                continue

            parts = line.split(",")

            # MUST be 7 values: 6 numbers + label
            if len(parts) != 7:
                continue

            try:
                features = list(map(float, parts[:6]))
                label = parts[6].strip()

                rows.append(features + [label])

            except:
                continue

    df = pd.DataFrame(rows, columns=["s1", "s2", "s3", "s4", "s5", "s6", "label"])
    return df
```

```
idle = load_file("/content/drive/MyDrive/Colab Notebooks/idle.txt")
shake = load_file("/content/drive/MyDrive/Colab Notebooks/shake.txt")
tilt = load_file("/content/drive/MyDrive/Colab Notebooks/tilt.txt")
```

```
data = pd.concat([idle, shake, tilt], ignore_index=True)
print(data.head())
print(data["label"].value_counts())
```

```
      s1      s2      s3      s4      s5      s6 label
0 -3880.0 -52.0 16876.0 276.0 38.0 -217.0 idle
1 -3872.0 -128.0 16900.0 293.0 44.0 -210.0 idle
2 -3864.0 -128.0 16864.0 269.0 74.0 -211.0 idle
3 -3856.0 -104.0 16836.0 268.0 35.0 -198.0 idle
4 -3912.0 -80.0 16892.0 260.0 38.0 -203.0 idle
label
shake    1542
tilt     1372
idle     1119
Name: count, dtype: int64
```

```
def create_features(df):

    # basic stats per row
    df["mean"] = df[["s1", "s2", "s3", "s4", "s5", "s6"]].mean(axis=1)
    df["std"] = df[["s1", "s2", "s3", "s4", "s5", "s6"]].std(axis=1)
    df["min"] = df[["s1", "s2", "s3", "s4", "s5", "s6"]].min(axis=1)
    df["max"] = df[["s1", "s2", "s3", "s4", "s5", "s6"]].max(axis=1)

    # RMS (important for IMU signals)
    df["rms"] = np.sqrt((df[["s1", "s2", "s3", "s4", "s5", "s6"]]**2).mean(axis=1))
```

```
return df
```

```
data = create_features(data)
```

```
X = data.drop("label", axis=1)
y = data["label"]
```

```
le = LabelEncoder()
y = le.fit_transform(y)
```

```
print(le.classes_)
```

```
['idle' 'shake' 'tilt']
```

```
WINDOW_SIZE = 50 # number of rows per window
STEP = 10 # how much we move each time
```

```
from scipy.stats import mode # Added import

def create_sliding_windows(X, y, window_size=50, step=10):
    X_windows = []
    y_windows = []

    X = X.values if hasattr(X, "values") else X
    y = np.array(y)

    for i in range(0, len(X) - window_size, step):
        window = X[i:i+window_size]

        # label = most frequent label in window
        label = mode(y[i:i+window_size], keepdims=True).mode[0]

        # flatten window into one vector
        X_windows.append(window.flatten())
        y_windows.append(label)

    return np.array(X_windows), np.array(y_windows)
```

```
from sklearn.model_selection import train_test_split

Xw, yw = create_sliding_windows(X, y, window_size=50, step=10)

xtrain, xtest, ytrain, ytest = train_test_split(
    Xw, yw,
    test_size=0.2,
    random_state=42,
    stratify=yw
)
```

```
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(xtrain, ytrain)
```

```
▼ RandomForestClassifier ⓘ ?
RandomForestClassifier(random_state=42)
```

```
ypred = model.predict(xtest)
ypred
```

```
array([2, 1, 1, 1, 1, 2, 1, 2, 0, 1, 0, 2, 2, 1, 2, 1, 1, 1, 0, 2, 1, 2,
       0, 2, 0, 0, 2, 2, 0, 2, 2, 1, 2, 0, 0, 1, 2, 0, 1, 2, 1, 1, 1, 0,
       1, 1, 1, 2, 0, 1, 2, 1, 0, 0, 1, 1, 1, 2, 0, 2, 1, 2, 0, 2, 2, 1,
       0, 1, 0, 2, 1, 0, 1, 2, 0, 2, 1, 1, 2, 0])
```

```
print("Accuracy:", accuracy_score(ytest, ypred))
print(classification_report(ytest, ypred))
```

```
Accuracy: 0.9875
      precision    recall  f1-score   support

0         1.00      0.95      0.98         22
```

1	0.97	1.00	0.98	31
2	1.00	1.00	1.00	27
accuracy			0.99	80
macro avg	0.99	0.98	0.99	80
weighted avg	0.99	0.99	0.99	80

```
print("Train score:", model.score(xtrain, ytrain))  
print("Test score:", model.score(xtest, ytest))
```

```
Train score: 1.0  
Test score: 0.9875
```

```
from sklearn.metrics import confusion_matrix  
print(confusion_matrix(ytest,ypred))
```

```
[[21  1  0]  
 [ 0 31  0]  
 [ 0  0 27]]
```